SnippetVault

Project walkthrough and interview preparation guide

🗺️ STOP 1 — The Big Picture

Let me start at the top: **what this project is and how it's structured**.

What is SnippetVault?

SnippetVault is a full-stack code snippet manager — think of it like a personal/team GitHub Gist with versioning, groups, folders, and a browser extension to save code from anywhere on the web.

It has **three separate sub-projects**:

```
SnippetVault-complete/
├── SnippetVault/              ← Spring Boot REST API (backend)
├── SnippetValut-Frontend/     ← React + Vite (web app)
└── SnippetValut-Extension/    ← Chrome Extension (MV3)
```

> 🔑 **Note the typo** — "SnippetValut" (without the e) on the frontend and extension dirs. Probably just a naming mistake from early in the project that stuck. Worth knowing in case an interviewer notices it in your repo.

Tech Stack at a Glance

Layer	Technology
Backend	Java 21, Spring Boot 3.5, Spring Security, Spring JDBC (JdbcTemplate), Spring Mail
Database	MySQL (raw SQL via JdbcTemplate — no JPA/Hibernate)
Frontend	React 18, Vite, React Router, Framer Motion, Context API
Extension	Chrome Extension Manifest V3 (vanilla JS)
Auth	Session-based (Spring Security form login + JSESSIONID cookie)
Password hashing	BCrypt

The Architectural Choice Worth Flagging 🚩

You used **JdbcTemplate (raw SQL)** instead of JPA/Hibernate. That's a deliberate trade-off:

- ✅ Full control over SQL, no N+1 problems, explicit query visibility
- ✅ Good for a project where you wanted to understand SQL deeply
- ❌ More boilerplate, no auto schema generation, you write every query by hand

Be ready for: *"Why didn't you use Spring Data JPA?"* — your answer is about control and explicitness.

Does this top-level picture feel right? Do you remember choosing JdbcTemplate specifically? Once you confirm, I'll move into the **Authentication & Security** feature in detail.

Good answer — and the version history + fork queries with multi-condition JOINs completely justify that choice. Keep that in mind.

🔐 STOP 2 — Authentication & Security

This is one of the most interview-heavy areas. Let me walk you through every piece.

How Auth Works (Session-Based)

You chose **session-based auth** — not JWT. Here's the flow:

1. User POSTs credentials to `/login` (email + password)
2. Spring Security validates against the DB, creates a server-side session
3. Browser receives a **`JSESSIONID` cookie** — all subsequent requests carry it
4. Spring Security reads that cookie, finds the session, and knows who's logged in
5. On logout, the session is **invalidated server-side** and the cookie is deleted

This is all configured in one place:
[SecurityConfig.java](file:///Users/diveshvalecha/Desktop/java/IdeaProjects/SnippetVault-complete/SnippetVault/src/main/java/com/divesh/SnippetVault/SecurityConfig.java)

SecurityConfig — The Key Decisions

1. Login uses email, not username

```
.usernameParameter("email")  // Override Spring's default "username"
```

Spring Security's form login defaults to a `username` field. You overrode it to `email` because that's your login identifier.

2. Custom JSON success/failure handlers

Instead of letting Spring redirect to HTML pages (its default behavior), you wrote inline lambda handlers that return **JSON responses**:

```
.successHandler((req, res, auth) -> {
    res.setStatus(SC_OK);
    res.getWriter().write("{\"message\" : \"Login Successful\", ...}");
})
```

This is critical because your frontend is a **separate SPA** — it needs JSON, not a redirect.

3. Unauthenticated request handler

```
.authenticationEntryPoint((request, response, authException) -> {
    response.setStatus(SC_UNAUTHORIZED);
    response.getWriter().write("{\"error\" : \"Please Login\"}");
})
```

Again — JSON instead of Spring's default login page redirect. This tells the frontend *"you're not logged in"* cleanly.

4. Public endpoints whitelist

```
.requestMatchers("/login", "/register", "/forgot-password",
                 "/validate-token", "/reset-password",
                 "/groups/public", "/abc123").permitAll()
```

Everything else requires authentication. The `/abc123` endpoint looks like a test/debug route — flag this if asked.

CORS Configuration — The Extension Integration

This is a **design decision worth talking about**:

```
configuration.setAllowedOrigins(Arrays.asList(
    "http://localhost:4200",   // Angular dev (leftover?)
    "http://localhost:3000",   // React CRA dev
    "http://localhost:5173",   // Vite dev (your actual frontend)
    "chrome-extension://bhpbjalgcgajnfjhmfbklikcoakchace"  // Your Chrome extension!
));
configuration.setAllowCredentials(true);  // ← Critical for cookies
```

You **explicitly whitelisted your Chrome extension's ID** in CORS — because the extension makes `fetch()` calls with `credentials: 'include'` to send the session cookie. Without `allowCredentials(true)` and an explicit origin, the browser blocks the cookie on cross-origin requests.

> 🚩 **Design flag:** The extension's Chrome ID is hardcoded. If you re-publish the extension, the ID changes and the backend breaks. A production system would need a config-based solution here.

CustomUserDetails — How the Logged-In User is Accessed Everywhere

You created [CustomUserDetails.java](file:///Users/diveshvalecha/Desktop/java/IdeaProjects/SnippetVault-complete/SnippetVault/src/main/java/com/divesh/SnippetVault/models/CustomUserDetails.java) which wraps Spring's `UserDetails` interface and **stores the user's DB `id`** alongside email and password.

Then [SecurityUtils.java](file:///Users/diveshvalecha/Desktop/java/IdeaProjects/SnippetVault-complete/SnippetVault/src/main/java/com/divesh/SnippetVault/utils/SecurityUtils.java) is a static helper:

```
public static CustomUserDetails getLoggedInUser() {
    Authentication auth = SecurityContextHolder.getContext().getAuthentication();
    return (CustomUserDetails) auth.getPrincipal();
}
```

This is called **in almost every service method** — `getMySnippets()`, `createSnippet()`, `deleteSnippet()`, etc. — so you never need to pass the user ID as a parameter through the controller layer. The controller just calls the service, and the service pulls the identity from the security context.

> 🚩 **Design flag:** This is a clean pattern, but it makes services harder to unit test in isolation (you'd need to mock the `SecurityContext`). Worth mentioning if asked about testing.

Forgot Password Flow

This is a full 3-step token-based reset — all in [AuthService.java](file:///Users/diveshvalecha/Desktop/java/IdeaProjects/SnippetVault-complete/SnippetVault/src/main/java/com/divesh/SnippetVault/service/AuthService.java):

1. **`/forgot-password`** — validates email exists → generates a UUID token → stores it in `auth_tokens` table with a **15-minute expiry** → emails a reset link
2. **`/validate-token`** — checks the token exists and hasn't expired (compared against `LocalDateTime.now()`)
3. **`/reset-password`** — validates token again → BCrypt-hashes the new password → updates the DB

Smart detail: before generating a new UUID, it checks if a **valid token already exists** for that user — so spamming "forgot password" doesn't create duplicate tokens:

```
String token = authRepository.checkIfTokenExists(user.getId(), currentTime);
if (token == null) {
    uuid = UUID.randomUUID().toString();  // Only create if none exists
} else {
    uuid = token;  // Reuse the existing valid token
}
```

> 🚩 **Design flag:** The email template still says *"Team Expense Tracker"* — it was copy-pasted from another project and not updated. Benign but worth knowing.

Does this all come back to you? Any part of the auth flow you want to dig into more?

Once you're good, we'll move to **STOP 3 — The Snippet System** (the core feature, with versioning, forking, and the visibility model).

✂️ STOP 3 — The Snippet System (Core Feature)

This is the heart of the project. There are several interlocking sub-features here — let me go one by one.

The Data Model

A `Snippet` has these fields (from [Snippets.java](file:///Users/diveshvalecha/Desktop/java/IdeaProjects/SnippetVault-complete/SnippetVault/src/main/java/com/divesh/SnippetVault/models/Snippets.java)):

```
snippetId, title, content, description, language
visibility       ← ENUM: 'PRIVATE' | 'PUBLIC' | 'GROUP'
currentVersion   ← integer, incremented on every edit
userId           ← owner
groupId          ← nullable (only set if visibility = GROUP)
folderId         ← nullable (for folder assignment)
activeYn         ← 0 or 1 (soft delete flag)
```

The **visibility ENUM** is the central access-control mechanism — it drives almost all query logic.

Visibility Model — Three Tiers

Visibility	Who can see it
PRIVATE	Only you
PUBLIC	Everyone (even unauthenticated, in theory)
GROUP	Only members of the linked group

Every "get snippets" query enforces this at the SQL level, not the Java level. For example, version history access check in [SnippetRepository.java](file:///Users/diveshvalecha/Desktop/java/IdeaProjects/SnippetVault-complete/SnippetVault/src/main/java/com/divesh/SnippetVault/repository/SnippetRepository.java#L125-L163):

```
WHERE sv.snippet_id = ?
AND (
    s.user_id = ?              -- you own it
    OR s.visibility = 'PUBLIC' -- it's public
    OR (
        s.visibility = 'GROUP'
        AND EXISTS (
            SELECT 1 FROM group_member gm
            WHERE gm.group_id = s.group_id AND gm.user_id = ?
        )
    )
)
```

This is exactly why you needed JDBC — this kind of conditional access query is awkward with JPA.

Versioning System — Git-lite for Snippets

This is the most technically interesting part. Every snippet has a **parallel `snippet_versions` table**.

On CREATE:

- Insert into `snippets` with `current_version = 1`
- Also insert into `snippet_versions` with `version_number = 1`

On EDIT (in `editSnippet`):

```
1. Read current_version from snippets table
2. newVersion = currentVersion + 1
3. INSERT new row into snippet_versions (the snapshot)
4. UPDATE the snippets row (title, content, current_version = newVersion)
```

So `snippets` always holds the **latest state**, and `snippet_versions` is an **append-only log** of every state the snippet has ever been in. You never delete version rows.

This gives you three powerful operations:

Endpoint	What it does
GET /snippets/{id}/versions	List all version summaries (number, timestamp, who edited)
GET /snippets/{id}/versions/{versionNumber}	View full content at that version
POST /snippets/{id}/restore/{versionNumber}	Restore — calls editSnippet with the old version's data, which itself creates a new version entry

> 🚩 **Design flag:** Restore doesn't roll back — it **creates a new version** with the old content. So your history is always forward-only. This is actually the right approach (same as Git revert vs reset), but be ready to explain it.

The `@Transactional` Annotation — Used Carefully

You annotated `createSnippet` and `editSnippet` with `@Transactional` in [SnippetService.java](file:///Users/diveshvalecha/Desktop/java/IdeaProjects/SnippetVault-complete/SnippetVault/src/main/java/com/divesh/SnippetVault/service/SnippetService.java#L29).

Why? Because each of those operations involves **two DB writes** (e.g., insert into `snippets` + insert into `snippet_versions`). If the second write fails, `@Transactional` rolls back the first — keeping data consistent. Without it, you could have a snippet with no version record, or a version record with no snippet.

Fork — The Most Interesting Endpoint

```
POST /snippets/{id}/fork/{versionNumber}
```

Here's what fork does:

1. Fetches the content of a **specific version** of a snippet (with full visibility checks — you can fork public or group snippets too)
2. Creates a **brand new snippet** owned by you, with `visibility = PRIVATE`, `groupId = null`
3. Calls `createSnippet()` internally — so the fork immediately starts its own version history at `v1`

> 🚩 **Design flag:** Fork is implemented by reusing `createSnippet()` — clean DRY code. The forked snippet is always private by default — that's a sensible safe default (you don't want to accidentally expose someone else's code). Be ready to explain that choice.

The `KeyHolder` Pattern — Getting the Auto-Increment ID Back

In `createSnippet`, after the INSERT you need the new `snippet_id` to immediately write to `snippet_versions`. You used Spring's `GeneratedKeyHolder`:

```
KeyHolder keyHolder = new GeneratedKeyHolder();
jdbcTemplate.update(connection -> {
    PreparedStatement ps = connection.prepareStatement(query,
Statement.RETURN_GENERATED_KEYS);
    // ... set params
    return ps;
}, keyHolder);
Long snippetId = keyHolder.getKey().longValue();
```

This is the standard JDBC pattern for retrieving a DB-generated primary key after an insert. The alternative (a second SELECT query) would be a race condition in concurrent scenarios.

Delete is Soft Delete

```
// deleteSnippet in SnippetRepository
"update snippets set active_yn = 0 where snippet_id = ? and user_id = ?"
```

You never actually DELETE rows. Every fetch query has `AND active_yn = 1`. This preserves version history and prevents orphaned foreign keys.

That covers the full snippet lifecycle. Does all of this ring a bell? Especially the versioning flow and the `@Transactional` usage?

Once you confirm, we'll move to **STOP 4 — Groups** (collaborative sharing with invite codes).

Viewed SecurityConfig.java:17-29

The `KeyHolder` Pattern — Deep Dive

The Problem It Solves

When you do an `INSERT` into a table with `AUTO_INCREMENT`, MySQL generates the primary key **inside the database**. Your Java code doesn't know what ID was assigned. But right after inserting into `snippets`, you **immediately need that `snippet_id`** to write the first row into `snippet_versions`.

You have two options:

Option	Problem
Run a second SELECT MAX(snippet_id)	Race condition — another insert could happen between your INSERT and SELECT
Use GeneratedKeyHolder	Gets the key back atomically in the same DB round-trip ✅

How It Works — Step by Step

Here's your actual code from [SnippetRepository.java](file:///Users/diveshvalecha/Desktop/java/IdeaProjects/SnippetVault-complete/SnippetVault/src/main/java/com/divesh/SnippetVault/repository/SnippetRepository.java#L39-L72):

```
// 1. Create an empty "box" to receive the generated key
KeyHolder keyHolder = new GeneratedKeyHolder();

// 2. Use the PreparedStatementCreator overload of jdbcTemplate.update()
jdbcTemplate.update(connection -> {

    // 3. Tell JDBC driver: after this INSERT, return generated keys
    PreparedStatement ps = connection.prepareStatement(
        query,
        Statement.RETURN_GENERATED_KEYS  // ← This flag is the key
    );

    ps.setString(1, title);
    ps.setString(2, content);
    // ... rest of params

    return ps;

}, keyHolder);  // ← Pass the box in here

// 4. After the INSERT completes, Spring reads the generated key
//    from the DB response and puts it into keyHolder
Long snippetId = keyHolder.getKey().longValue();

// 5. Now immediately use it for the second INSERT
jdbcTemplate.update(query2, snippetId, 1, title, content, ...);
```

What Actually Happens Under the Hood

When you pass `Statement.RETURN_GENERATED_KEYS` to `prepareStatement()`, JDBC tells MySQL:

> *"After you execute this INSERT, send me back the auto-generated row ID in the same response."*

MySQL returns it in the response packet itself. Spring's `GeneratedKeyHolder` intercepts that response and stores the key. No second query ever hits the DB.

Why the Lambda Syntax?

The normal `jdbcTemplate.update(sql, args...)` overload is simpler, but it **doesn't let you set `RETURN_GENERATED_KEYS`** — that flag has to be set at the `PreparedStatement` creation level, which only the `PreparedStatementCreator` lambda form gives you access to.

That's why `createSnippet` looks more verbose than all your other repository methods — it's the only one that needs the generated key back.

The Interview Answer (30 seconds)

> *"After inserting a snippet, I immediately needed its auto-incremented ID to create the first version record. I used Spring's `GeneratedKeyHolder` with a `PreparedStatementCreator` lambda — passing `RETURN_GENERATED_KEYS` to the `PreparedStatement`. This tells MySQL to return the generated ID in the same response as the INSERT, so I get it atomically without a race-condition-prone second query."*

Ready to move on to **STOP 4 — Groups**?

👥 STOP 4 — Groups (Collaborative Sharing)

Groups are the social layer of SnippetVault — they let multiple users share snippets with each other through an invite-code system.

The Data Model — Three Tables Working Together

```
user_groups                    group_member
_____________________          ____________________________
group_id (PK)                  id (PK)
name                           user_id  ──→ users
description                    group_id ──→ user_groups
group_code  (unique)           active_yn
group_visibility (PUBLIC|PRIVATE)
user_id (creator/owner)
```

Key constraint in `group_member`:

```
UNIQUE (user_id, group_id)
```

This prevents a user from joining the same group twice at the DB level — a safety net on top of the application-level check.

The Invite Code System

When you create a group, a **6-character alphanumeric code** is randomly generated in [GroupService.java](file:///Users/diveshvalecha/Desktop/java/IdeaProjects/SnippetVault-complete/SnippetVault/src/main/java/com/divesh/SnippetVault/service/GroupService.java#L53-L60):

```
private static final String CHARACTERS = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
private static final SecureRandom random = new SecureRandom();

do {
    groupCode = generateGroupCode(6);
} while (groupRepository.checkIfCodeUsed(groupCode) != null);
```

Two things worth noting:

1. **`SecureRandom` not `Random`** — cryptographically secure randomness. Important because this code acts as an access credential.
2. **Collision-avoidance loop** — keeps regenerating until it finds a code that doesn't already exist in the DB. With 36^6 = 2.17 billion combinations, collisions are practically impossible, but the loop makes it bulletproof.

Group Lifecycle — The Full API

Endpoint	What it does
POST /group	Create group → auto-joins creator (two operations, one call)
POST /join-group/{groupCode}	Join by invite code
GET /groups/public	List all PUBLIC groups (no auth required — it's whitelisted)
GET /groups/my-groups	List groups you've joined
DELETE /leave-group/{groupCode}	Leave a group

Smart detail in `createGroup` — the controller calls two service methods back to back:

```
String groupCode = groupService.createGroup(group);
groupService.joinGroup(groupCode);  // Creator auto-joins their own group
```

The creator is automatically added as a member. Without this, the creator would have visibility set to `GROUP` on their own snippets but couldn't access them.

> **Design flag:** The creator has no special "admin" role — they're just a regular member like anyone else. There's no way to kick members or transfer ownership. It's a minimal viable groups system.

Access Control at Query Time

When you try to post a snippet to a group, the service verifies membership **before** the DB write:

```
Integer res = groupRepository.checkIfAlreadyJoined(groupId, userId);
if (res == null) {
    throw new GroupDoesnotExits("User has not joined this group");
}
```

And when fetching group snippets, same check:

```
Group group = groupRepository.checkIfCodeUsed(groupCode);
if (group == null)       throw new GroupDoesnotExits();
Integer res = groupRepository.checkIfAlreadyJoined(groupId, userId);
if (res == null)         throw new GroupDoesnotExits("User has not joined this group");
return snippetRepository.getGroupSnippets(groupId, userId);
```

So there are two gates: the service-level membership check AND the SQL-level visibility filter.

The Public Groups Endpoint — One Quirk

```
// GroupController
@GetMapping("/groups/public")
public ResponseEntity<Map<Group, Integer>> getPublicGroups(){
    Map<Group, Integer> test = groupService.getPublicGroups();
    return ResponseEntity.ok().body(test);
}
```

The return type is `Map<Group, Integer>` — a Group object mapped to a member count. The variable is named `test` and the log says `"reached controller"` / `"reached service"` — this is clearly some debug code left in from development.

> **Design flag:** These leftover debug log statements and the variable name `test` are worth cleaning up for production, but they signal the natural iterative development process. If asked, just be honest — *"this was during development, I was verifying the data shape before cleaning it up."*

Leave Group — How It Works

```
// leaveGroup in GroupService
Integer res = groupRepository.checkIfAlreadyJoined(id, userId);
if (res != null) {
    groupRepository.leaveGroup(res);  // res is the group_member.id
}
```

Note: `checkIfAlreadyJoined` returns the `group_member.id` (the join row's PK), not just a boolean. So leaving a group is a soft delete on that specific row by its ID:

```
UPDATE group_member SET active_yn = 0 where id = ?
```

Consistent with the soft delete pattern used throughout the entire project.

Does the groups system make sense? Remember the invite code flow and the double-gate access control — those are likely interview talking points.

Next up: **STOP 5 — Folders** (snippet organization), which is shorter. Then we'll hit the **Frontend** and **Chrome Extension** before the final summary.

Viewed GroupService.java:22-38

📁 STOP 5 — Folders (Snippet Organization)

Folders are the lightest feature — essentially a tagging/organization layer on top of snippets.

The Data Model — Two Tables

```
folders                          folder_snippets
─────────────────────            ──────────────────────────
folder_id (PK)                   id (PK)
name                             folder_id ──→ folders
description                      snippet_id ──→ snippets
user_id  ──→ users               UNIQUE(folder_id, snippet_id)
group_id (nullable, future use)
active_yn
```

The join table `folder_snippets` is a **many-to-many** relationship — one snippet can be in multiple folders, one folder can contain multiple snippets. The `UNIQUE(folder_id, snippet_id)` constraint prevents duplicates at the DB level.

> 🚩 **Design flag:** `folders.group_id` exists in the schema with a comment `-- optional (for shared folders later)`. This is an **unimplemented future feature** — the column is there but nothing in the code uses it yet. Shows forward-thinking schema design.

The Full API

Endpoint	What it does
POST /folder	Create a folder
GET /folder	List your folders
DELETE /folder/{id}	Soft delete a folder
POST /folder/{id}/snippet/{snippetId}	Add snippet to folder
GET /folder/{id}/snippet	Get all snippets in a folder

Endpoint	What it does
DELETE /folder/{id}/snippet/{snippetId}	Remove snippet from folder

Clean, RESTful, resource-scoped URLs.

How Folder Assignment Works at Snippet Creation

This is a subtle but important flow in [SnippetService.java](file:///Users/diveshvalecha/Desktop/java/IdeaProjects/SnippetVault-complete/SnippetVault/src/main/java/com/divesh/SnippetVault/service/SnippetService.java#L49-L53):

```
Long snippetId = snippetRepository.createSnippet(...);

if (snippet.getFolderId() != null) {
    folderService.addSnippetToFolder(snippet.getFolderId(), snippetId);
}
```

When creating a snippet, you can optionally pass a `folderId` in the request body. If present, the service:

1. First creates the snippet (gets back the new `snippetId` via `KeyHolder`)
2. Then links it to the folder in `folder_snippets`

Both happen inside the same `@Transactional` method — so if the folder assignment fails, the snippet creation rolls back too.

The same pattern repeats in `editSnippet` — you can reassign a snippet to a different folder during an edit.

One Design Decision Worth Noting

[FolderController.java](file:///Users/diveshvalecha/Desktop/java/IdeaProjects/SnippetVault-complete/SnippetVault/src/main/java/com/divesh/SnippetVault/controller/FolderController.java) has **two separate ways** to add a snippet to a folder:

1. **At creation time** — via `folderId` field in the snippet POST body (handled by `SnippetService`)
2. **Explicitly after creation** — via `POST /folder/{id}/snippet/{snippetId}` (handled by `FolderController` directly)

This gives the frontend flexibility — create now, organize later.

Frontend Folder Behavior

In [SnippetContext.jsx](file:///Users/diveshvalecha/Desktop/java/IdeaProjects/SnippetVault-complete/SnippetValut-Frontend/src/context/SnippetContext.jsx#L145-L163), selecting a folder triggers a dedicated fetch:

```
useEffect(() => {
    if (!selectedFolderId) {
        setSelectedFolderSnippets([]);
        return;
    }
    fetchFolderSnippetsApi(selectedFolderId);
}, [selectedFolderId]);  // ← Reacts to folder selection
```

So folder snippets are fetched **lazily** — only when you click a folder, not all upfront. Smart for performance.

Folders are short but they complete the organizational picture:

```
User → owns → Snippets (visibility: private/public/group)
User → owns → Folders → contain → Snippets (many-to-many)
User → joins → Groups → associated with → Snippets
```

Moving on — STOP 6 is the Frontend (React app). This one covers how state is managed across the whole app, the routing, and how it all connects back to the Spring backend. Ready?

⚛️ STOP 6 — Frontend (Quick Overview)

React + Vite SPA. Two pages: Auth (login/signup/forgot-password) and Dashboard.

State is managed via **three React Contexts** nested in `App.jsx`:

- `AuthContext` — user session, stored in `localStorage`
- `GroupContext` — joined groups
- `SnippetContext` — all snippets + folders, fetched and deduplicated on load

Protected routing via a `PrivateRoute` wrapper — redirects to `/login` if no user in context.

Uses **Framer Motion** for page transition animations.

The frontend calls the Spring backend at `localhost:8080` using `credentials: 'include'` on every fetch so the session cookie is sent automatically.

That's it — frontend is thin on logic because the backend does all the heavy lifting.

STOP 7 next — the Chrome Extension, which is actually one of the more unique/impressive parts of the project. Ready?

🧩 STOP 7 — Chrome Extension (Quick Overview)

A **Manifest V3** Chrome extension with three parts:

- **`content.js`** — injected into every webpage, listens for `Ctrl+Shift+S` (or `Cmd+Shift+S` on Mac) to capture selected text
- **`background.js`** — service worker that adds a right-click context menu option *"Save to Snippet Vault"*, stores the selected code in `chrome.storage.session`
- **`popup/`** — the UI that opens when you click the extension icon. Checks if you're logged in (by hitting the backend), auto-detects language from the code, and lets you save it as a snippet directly to your vault with folder/group/visibility options

Two ways to trigger it: **right-click → context menu** or **keyboard shortcut**. Either way the selected code gets pre-filled in the popup form.

Reuses the same backend session cookie as the web app — so if you're logged in on the frontend, the extension just works.

Ready for the **Final Summary** — the one-liner pitch, feature list, tech stack, and the 3 key design decisions you should be ready to explain in an interview?

🏁 Final Summary — Interview Cheat Sheet

One-Line Pitch

> *"SnippetVault is a full-stack code snippet manager with version history, team groups, folder organization, and a Chrome extension to save code from any webpage — built with Spring Boot, MySQL, React, and a Chrome MV3 extension."*

Core Features

- ✅ Email/password auth with session cookies + forgot-password via tokenized email
- ✅ Create, edit, delete snippets with `PRIVATE / PUBLIC / GROUP` visibility
- ✅ **Full version history** on every snippet — view, restore, or fork any past version
- ✅ **Fork** — copy any accessible snippet version into your own private snippet
- ✅ **Groups** — invite-code based collaborative spaces for sharing snippets
- ✅ **Folders** — organize snippets into personal collections
- ✅ **Chrome Extension** — save code from any webpage via right-click or keyboard shortcut

Tech Stack

Backend	**Java 21, Spring Boot 3.5, Spring Security, JdbcTemplate, Spring Mail**
Database	MySQL (raw SQL, no ORM)
Frontend	React 18, Vite, React Context API, Framer Motion
Extension	Chrome Manifest V3, vanilla JS
Auth	Session-based (JSESSIONID cookie), BCrypt passwords

3 Design Decisions to Be Ready For

1. "Why JdbcTemplate over JPA?"

> Complex multi-condition JOIN queries (especially the visibility access checks across `snippets`, `group_member`, and `snippet_versions`) are cleaner and more explicit in raw SQL. JPA would obscure the logic and make debugging harder. Also wanted deep understanding of the Spring-MySQL connection layer.

2. "How does your versioning work?"

> Append-only `snippet_versions` table. Every create/edit writes a new version row. `snippets` table always holds the latest state. Restore doesn't roll back — it creates a new version with old content (like `git revert`, not `git reset`). `@Transactional` ensures both writes succeed or both roll back.

3. "How does the Chrome extension authenticate with the backend?"

> It reuses the existing browser session cookie (`JSESSIONID`). The backend's CORS config explicitly whitelists the extension's Chrome ID and sets `allowCredentials: true`. So if you're already logged in on the web app, the extension is authenticated automatically — no separate login needed.

Whenever you're ready, just say **"switch to interviewer mode"** and I'll start grilling you. 🎯